

第十章 软件测试

本章速览

核心问题：软件测试是以最少的人力、物力和时间找出软件中潜在的各种错误和缺陷，并证明软件的功能和性能与需求说明相符。**测试不能证明软件中没有错误，只能说明软件中存在错误**（Dijkstra：测试只能证明缺陷存在，不能证明缺陷不存在）。

两条主线：① **白盒测试**（结构测试，看内部逻辑）—逻辑覆盖 + 基本路径测试，单元测试阶段为主；② **黑盒测试**（功能测试，看外部规格）—等价类划分 + 边界值分析 + 因果图，集成/确认/系统/验收测试为主。

测试阶段（必背 5 步）：单元测试 → 集成测试 → 确认测试 → 系统测试 → 验收测试。

高频大题：基本路径法（画控制流图 → 算 V(G) → 写独立路径集）、等价类划分（列表 → 设计用例）。

题目关键词导航

关键词	指向	关键词	指向	关键词	指向
测试目的/证明正确	测试 = 发现错误，非证明正确	白盒 / 黑盒	测试方法分类	语句/判定/条件覆盖	逻辑覆盖 6 准则
V(G)、环路复杂度	基本路径测试，3 种公式	独立路径集	基本路径测试	控制流图 / 复合条件拆分	流程图→控制流图
等价类（有效/无效）	等价类划分	边界值	边界值分析	因果图 / 判定表	因果图法
驱动模块 / 桩模块	单元测试环境	自顶向下 / 自底向上	集成测试策略	α 测试 / β 测试	验收测试
Error/Defect/Fault/Failure	术语区分	穷举测试不可能	测试原则	回归测试	测试流程

知识主线

测试是什么（定义/目的/原则/对象）→ 测试分几种（白盒/黑盒；单元/集成/确认/系统/验收）→ 黑盒怎么测（等价类 → 边界值 → 因果图）→ 白盒怎么测（逻辑覆盖 6 准则 → 基本路径测试）→ 5 个测试阶段（含 α/β、驱动/桩、自顶向下/自底向上）→ 测试流程与回归测试 → 术语区分（Error/Defect/Fault/Failure）→ 可靠性（MTTF）。

一、核心知识点详解（PPT 主线）

1.1 软件测试的目的（必考）

从用户角度：通过测试暴露软件中的错误和缺陷，以考虑是否可接受该软件产品。

从开发者角度：希望测试成为表明软件产品中不存在错误的过程，验证软件已正确实现用户要求，确立对软件质量的信心。

Myers 经典观点（背）：

- 测试是程序的执行过程，目的在于发现错误；
- 一个好的测试用例在于能发现至今未发现的错误；
- 一个成功的测试是发现了至今未发现的错误的测试；
- 测试不能表明软件中不存在错误，只能说明软件中存在错误。**

定义：以最少的时间和人力，系统地找出软件中潜在的各种错误和缺陷，并证明软件的功能和性能与需求说明相符合。

★考点 "测试目的 = 发现错误" 几乎每年必考（判断/单选/简答）。凡出现"证明正确 / 没有错误 / 找出所有错误"等表述，一律判错。

1.2 软件测试的原则（必考）

- 应当把"尽早地和不断地进行软件测试"作为座右铭；
- 测试用例**应由**测试输入数据**和**对应的预期输出结果**这两部分组成；
- 程序员应避免检查自己的程序（心理定势影响）；
- 设计测试用例时，应当包括合理的输入条件和不合理的输入条件；
- 充分注意测试中的**群集现象**：残存错误数目与已发现错误数目成正比；
- 严格执行测试计划，排除测试随意性，对每一个测试结果做全面检查；
- 妥善保存测试计划、测试用例、出错统计和最终分析报告。

补充原则（教材）：所有测试都应以用户需求为依据；穷举测试不可能，须终止；测试不能表明软件中不存在错误。

1.3 软件测试对象

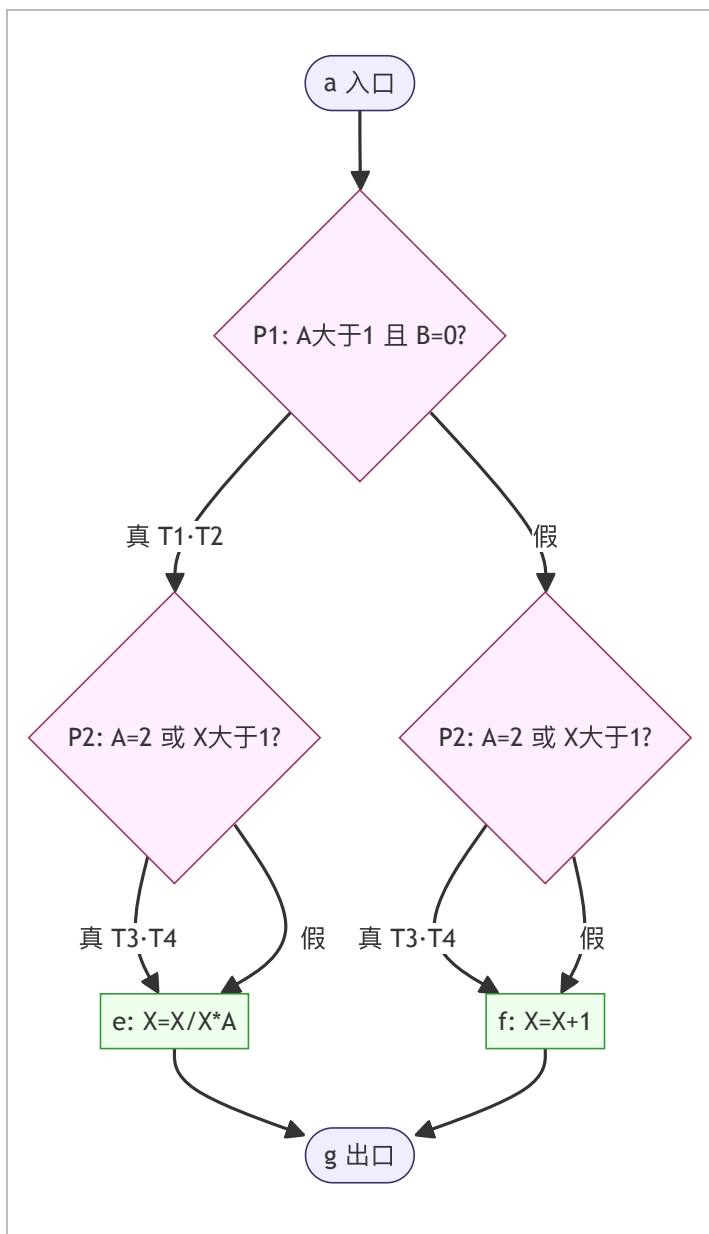
软件测试 ≠ 程序测试。测试应贯穿软件定义与开发整个周期。需求规格说明、概要设计、详细设计、源程序都是测试对象。文档+代码均为测试对象。

1.4 黑盒测试 vs 白盒测试（必考对比）

项目	白盒测试	黑盒测试
别名	结构测试 / 逻辑驱动测试	功能测试 / 数据驱动测试
是否看内部	把程序看作透明盒子，利用程序内部逻辑结构	完全不考虑内部逻辑与内部特性
设计依据	详细设计说明书 + 源程序	需求规格说明 + 概要设计说明
主要方法	逻辑覆盖（6 准则）+ 基本路径测试	等价类划分、边界值分析、因果图、错误推测、判定表
适用阶段	主要单元测试	集成 / 确认 / 系统 / 验收测试
检查重点	程序模块所有独立执行路径；逻辑判定真假；循环边界；内部数据结构	功能是否正确；接口输入输出；数据结构错误；性能；初始化/终止性错误

1.5 白盒测试 — 逻辑覆盖 6 准则（极高频）

示例程序流程图（两个复合判定）：第一判定 (A>1) and (B=0)；第二判定 (A=2) or (X>1)。条件标记：T1=A>1, T2=B=0, T3=A=2, T4=X>1；取假加横线上标。



四条路径：L1=ace；L2=abd；L3=abe；L4=acd。

覆盖准则	要求	强弱	缺陷
语句覆盖	每条可执行语句至少执行 1 次	最弱	对逻辑运算符错误（如 and↔or）不敏感
判定覆盖 (分支覆盖)	每个判定的取真/取假分支各 ≥1 次	稍强	不能保证查出条件内部错误（如 X>1 写成 X<1）
条件覆盖	每个判定中每个条件的可能取值各 ≥1 次	不一定包含判定覆盖	可能只覆盖部分分支
判定-条件覆盖	既满足条件覆盖又满足判定覆盖	更强	某些条件会被短路求值掩盖（and/or 的短路）
条件组合覆盖 (多重条件覆盖)	每个判定所有条件取值组合各 ≥1 次	更强	可能遗漏某些路径（如 L4 acd 缺失）
路径覆盖	覆盖程序中所有可能路径	最强（路径维度）	对循环程序不现实；不一定覆盖所有条件组合

强弱排序（背）：语句 < 判定 < 条件 < 判定-条件 < 条件组合；路径覆盖与条件组合覆盖侧重点不同，通常综合多重条件覆盖与路径覆盖设计更完备的测试用例。

★考点 大题常用模板：分别写出满足 6 种覆盖准则的测试用例集合（输入 A、B、X 与输出 A、B、X）。注意“条件组合覆盖”需先拆解复合条件再列举组合。

1.6 基本路径测试（应用大题必考）

适用场景：循环结构使路径数爆炸 → 把覆盖路径数压缩到环路复杂度 $V(G)$ 以内，循环体最多执行 1 次。

① 程序流程图 → 控制流图（CFG）转换规则

- 顺序结构的多个结点可合并为一个结点；
- 在选择/多分支结构中，分支汇聚处应有虚拟汇聚结点；
- 边和结点圈定的范围 = 区域，图形外也算 1 个区域；
- 复合条件必须拆成单条件嵌套判定（AND/OR/NAND/NOR 连接的复合条件 → 一系列只有单个条件的嵌套判定）。

■ 图示 复合条件拆分示例：(A>1) and (B=0) 在控制流图中需拆成两个串行判定结点 — 先判 A>1，再判 B=0，两者都真才走“真”分支；(A=2) or (X>1) 拆为：先判 A=2，真则整体真；假则再判 X>1。

② 环路复杂度 $V(G)$ — 三种计算公式（背）

1. 区域数法： $V(G) = \text{控制流图中区域数（含开放区域）}$ ；
2. 欧拉公式： $V(G) = E - N + 2$ ，E=边数，N=结点数；
3. 判定结点法： $V(G) = P + 1$ ，P=判定结点数（拆分后）。

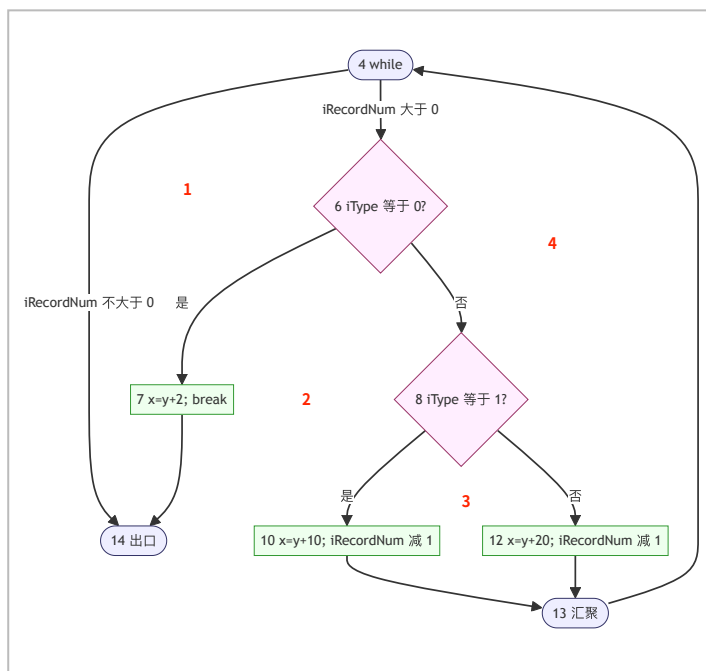
③ 基本路径集

从入口到出口的路径，每条至少经历一条从未走过的边。基本路径集不唯一，最大条数 = $V(G)$ 。

④ 教材标准例题（必会）

代码：

```
void Func(int iRecordNum, int iType){
    int x=0,y=0;
    while(iRecordNum > 0){           // 4
        if(0 == iType){ x=y+2; break;} // 6→7
        else if(1 == iType){ x=y+10; iRecordNum--; } // 8→10
        else { x=y+20; iRecordNum--; } // 12
    } // 13
} // 14
```



三种公式验算：① 区域数=4；② $E = 8, N = 10$ （按教材编号）， $V(G)=E-N+2=4$ ；③ 判定结点 $P=3$ （4、6、8）， $V(G)=P+1=4$ 。

独立路径集（4 条）：

- 路径1：4 → 14 输入 iRecordNum=0；预期 x=0
 路径2：4 → 6 → 7 → 14 输入 iRecordNum=1，iType=0；预期 x=2
 路径3：4 → 6 → 8 → 10 → 13 → 4 → 14 输入 iRecordNum=1，iType=1；预期 x=10
 路径4：4 → 6 → 8 → 12 → 13 → 4 → 14 输入 iRecordNum=1，iType=2；预期 x=20

★考点 评分要点（历年阅卷标准）：① 关键判定结点必须出现，少 1 个扣 0.5；② $V(G)$ 必须给出至少一种计算方法说明，否则扣 0.5~1；③ 独立路径条数 ≤ $V(G)$ ，每条至少包含 1 条其他路径未走过的边，错 1 条扣 0.5。

1.7 等价类划分（应用大题必考）

思想：不能穷举 → 找有代表性的等价类，测试其代表值 = 测试该类全部值。

- 有效等价类：对规格说明合理、有意义的输入集合；
- 无效等价类：对规格说明不合理、无意义的输入集合。

划分原则（5 条）

输入条件类型	有效等价类	无效等价类
区间 [x, y]	1 个 ($x \leq v \leq y$)	2 个 ($< x$ 、 $> y$)
数值集合	1 个 (集合内)	1 个 (集合外)
布尔量	1 个 (真)	1 个 (假)
一组离散值且分别处理	每个值各 1 个	1 个 (所有不允许值)
规则/限制	1 个 (完全符合)	若干 (从不同角度违反规则)

测试用例设计原则（背）

- 给每个等价类唯一编号；
- 设计测试用例，尽可能多地覆盖尚未覆盖的有效等价类（1 个用例覆盖多个），重复直至有效等价类全覆盖；
- 设计测试用例，每次仅覆盖 1 个尚未覆盖的无效等价类，重复直至无效等价类全覆盖。

★考点 公式：若 V 个有效、IV 个无效等价类，最少用例数 = 1 + IV（有效尽量合并，无效必须单测）。如 3 个有效 + 4 个无效 → 5 组。

等价类表标准结构

输入条件	有效等价类	编号	无效等价类	编号
字段 1	...	1	...	n
字段 2	...	...	...	...

1.8 边界值分析

核心：大量错误发生在输入/输出范围的边界上，而非内部。是等价类划分的补充。

取值原则：选正好等于、刚刚大于、刚刚小于边界的值。

- 输入范围 [x, y]：取 x、x+ε、y、y-ε，以及越界值 x-ε、y+ε；
- 输入个数（最少 n、最多 m）：取 n-1、n、n+1、m-1、m、m+1；
- 有序集合：取第一个与最后一个元素。

三角形判别反例：A+B>C 等 6 个条件，“>”错写成“≥”即不能构成三角形 — 边界值能查出此类错误。

1.9 因果图法（结合判定表）

解决的问题：等价类/边界值只考虑单个输入，无法覆盖多条件组合。因果图 = 检查多输入条件各种组合下系统是否有问题。

步骤（背）

- 分析规格说明，分出原因 Ci（输入或状态）与结果 Ei（输出或动作），赋唯一标识符；
- 分析语义，找出原因-原因、原因-结果关系，画因果图；
- 在图上标注约束条件（E/I/O/R/M）；
- 把因果图转换成判定表，按约束去掉不可能组合；
- 判定表每一列 = 一个测试用例。

基本关系符号

关系	含义
恒等 ≡	原因出现则结果出现
非 ¬	原因出现则结果不出现
或 ∨	几个原因中有一个出现则结果出现
与 ∧	几个原因都出现结果才出现

约束符号

约束	含义
E（互斥/排他）	a、b 两个原因不同时成立
I（包含/或）	a、b、c 中至少一个成立
O（唯一）	a、b 中必须且仅一个成立
R（要求）	a 出现时 b 必须出现
M（屏蔽）	a=1 时 b 必须=0；a=0 时 b 不定

自动售货机经典例（5 角/1 元，橙汁/啤酒，零钱找完灯）

原因 C1=有零钱找、C2=投 1 元、C3=投 5 角、C4=按橙汁、C5=按啤酒；约束：C2 与 C3 互斥（E）、C4 与 C5 互斥（E）。结果 E21=零钱找完灯亮、E22=退 1 元、E23=退 5 角、E24=出橙汁、E25=出啤酒。中间结点 11/12/13/14 表示投入并按钮、应找零钱等。

1.10 五个测试阶段（必考大题）

① 单元测试（Unit Testing）

- 对象：最小单位（程序模块、对象、类、函数、组件）；
- 依据：详细设计说明书；
- 方法：白盒为主，黑盒为辅；
- 5 个测试方面：模块接口测试、局部数据结构测试、独立路径测试、错误处理测试、边界测试；
- 辅助模块：驱动模块 driver（相当于主程序，接收数据传给被测模块；桩模块 stub（存根模块）代替被测模块调用的子模块；
- 人员：开发人员为主，测试人员为辅。

② 集成测试（Integration Testing）

- 对象：已通过单元测试、按概要设计组装起来的模块群；
- 依据：概要设计说明书；
- 方法：黑盒为主，关注接口；
- 验证问题：接口数据是否丢失；模块间副作用；子功能组合能否达到父功能；全局数据结构；误差累积放大。

三种集成策略对比：

策略	特点	优点	缺点
一次性集成（大棒集成）	所有单元一次性组装	无需驱动/桩模块	问题多、定位难，一般不用
自顶向下	从主模块沿控制层次向下	早期验证主要控制/判断点；无须驱动模块；可深度/广度优先	需大量桩模块；底层关键单元问题后期才暴露；不利并行
自底向上	从底层模块组向上集成	可并行；早发现底层关键问题；无须桩模块	需驱动模块；系统最后才能运行，顶层验证推迟
三明治集成（混合）	上层自顶向下 + 下层自底向上	两端并行；系统早可运行	需桩+驱动模块；组织协调复杂

③ 确认测试（Validation / 有效性测试）

- 对象：已完成集成测试、基本可运行的系统；
- 依据：需求规格说明书 SRS（这是判断题高频陷阱！）；
- 方法：黑盒测试；
- 人员：开发组织中专门的测试人员；
- 附加：软件配置复查；可移植性、兼容性、出错自动恢复、可维护性等非功能需求测试。

④ 系统测试

- 对象：部署到真实运行环境的整个系统（硬件/网络/OS/DB/外围系统/用户）；
- 依据：需求规格说明 + 用户合同 + 运行环境约束；
- 包含：功能、性能、压力/强度、疲劳强度、大数据量、安全性、易用性、兼容性、健壮性、互联、安装、启停、配置、文档测试。

⑤ 验收测试（Acceptance Testing）

- 对象：试运行一段时间后稳定的系统；面向用户的核心功能；
- 人员：以用户为主，开发者、QA 配合；
- 用例：粗粒度、结构简单、不过多描述内部实现；
- 面向大众产品：α 测试 + β 测试。

1.11 α 测试 vs β 测试

维度	α 测试	β 测试
环境	开发环境（受控）	实际使用环境（用户各自计算机）
是否有开发者在场	开发者可坐在用户旁边记录	用户独立使用，自行记录并报告
人员	除开发小组外首批用户	更广大的用户群体
目的	评价 FURPS（功能、可使用性、可靠性、性能、支持）	发现只有最终用户才能发现的错误
时序	先 α，需软件达到一定稳定性	α 达到可靠程度后发布 β 版
阶段	验收测试前期	整个测试的最后阶段

1.12 软件测试专业术语（必考对比）

术语	含义	视角
Error 错误	常说的 bug；人在需求分析、设计和开发中人为产生的错误，可存在于文档或代码中	面向开发（人为差错）
Defect 缺陷	软件（文档/数据/程序）中不希望或不可接受的偏差	静态状态
Fault 故障	运行过程中产生的不可接受的系统内部状态	动态行为（缺陷被激活）
Failure 失效	运行过程中产生的不可接受的系统外部状态	外部表现（故障未容错即失效）

二、补充知识点（教材独有）

2.1 软件测试 H 模型

测试过程与软件开发其他过程并行执行。上分支=测试过程，下分支=开发其他过程。任一待测内容准备就绪即可开展测试（不必全部完成才测）。

2.2 广义测试三活动：确认、验证、测试

- 确认 (Validation)：是否构造了正确的软件（与用户期望一致）；
- 验证 (Verification)：是否正确地构造了软件（每阶段结果与前一阶段一致）；
- 测试：设计用例、运行程序、执行系统测试。

2.3 软件测试分类（多角度）

- 按开发阶段：单元 / 集成 / 确认 / 系统 / 验收；
- 按测试对象：需求 / 设计 / 代码 / 文档测试；
- 按组织方式：开发方 / 用户 / 第三方测试；
- 按用例设计技术：白盒 / 黑盒；
- 按是否运行：静态 / 动态测试；
- 按执行方式：手工 / 自动化 / 半自动化；
- 按内容：功能 / 性能 / 安全性 / 易用性 / 兼容性测试。

2.4 软件可测试性（表 10-3）

程序能被测试的容易程度。影响因素：可用性、可观察性、可控制性、可分解性、简单性、稳定性、易理解性。

2.5 软件测试流程（7 步）

- 组建测试团队（测试经理、设计人员、工程师、配置人员）；
- 制订测试计划（输出《软件测试计划》）；
- 设计测试用例（输出《软件测试用例》，需评审 + 配置管理）；
- 搭建测试环境（硬件、软件、数据环境）；
- 执行测试及结果记录（输出《软件测试错误记录》）；
- 错误修改（New → Open → Fixed → Closed / ReOpen / Declined / Deferred）；
- 回归测试（全回归 / 基于风险 / 基于操作频度 / 基于影响分析）；
- 测试总结（输出《软件测试总结报告》）。

2.6 回归测试

软件发生变动时确保修改有效性：① 修改是否符合要求（缺陷正确修正、新功能正确运行）；② 是否对原有功能造成不利影响。回归测试包策略：全回归、基于风险、基于操作频度、基于影响分析。

2.7 错误状态机

因果链：Error（人为差错）→ Defect（静态偏差）→ Fault（运行时内部异常）→ Failure（外部失效）。测试通过 Failure 反推定位 Error，修复 Defect。

状态	含义
New	新报告的错误
Open	已确认并分配处理中
Fixed	已修正，等待回归测试
Declined	不是错误，拒绝修改
Deferred	延期到后续版本
ReOpen	回归测试仍存在问题
Closed	已修复并验证关闭

2.8 自动化测试

原理：捕获/回放用户操作 → 转换为脚本 → 加入验证点 → 自动运行对比。

不适用场景：不稳定系统；主观感受型测试（易用性、文档）；开发周期短的系统。

测试工具分类：白盒工具、黑盒工具、测试管理工具、静态/动态分析工具、测试数据自动生成工具、模块测试台、集成测试环境。

- 单元：C++Test、JUnit、JTest；
- GUI：Rational Robot、WinRunner、SilkTest；
- 负载性能：LoadRunner、QALoad、WebLoad、PerformanceStudio；
- 缺陷跟踪：ClearQuest、TrackRecord、Bugzilla。

2.9 软件可靠性

可用性：给定时刻 t 系统成功运行的概率。可靠性：给定时间间隔 [0,t] 内系统未失效的概率。

MTTF（平均无故障时间，Shooman 模型）：

$$MTTF = \frac{1}{K [E_T/I_T - E_c(t)/I_T]}$$

其中 K≈200 经验常数； E_T 测试前程序原有故障总数； I_T 程序长度；t 测试时间； $E_c(t)$ 0~t 内检出并排除故障数。

估算 E_T 方法：① Shooman 瞬间估算法（两次独立测试列方程组）；② 植入故障法（捕获-再捕获抽样） $\hat{N}_0 = nN_s/n_s$ ；③ Hyman 分别测试法 $B_0 = B_1B_2/b_c$ 。

2.10 软件错误分类（教材表 10-2，简答备用）

- 按严重程度：严重 / 较严重 / 一般 / 建议；
- 按引入阶段：需求定义类 / 规格说明类 / 框架设计类 / 算法设计类 / 编码类；
- 按性质：数据错误 / 加工错误 / 显示错误；
- 按接口层次：硬件接口 / 操作系统接口 / 数据库接口 / 系统间接口 / 模块间接口 / 模块内接口。

三、核心对比表（背）

3.1 黑盒 vs 白盒

	白盒	黑盒
视角	透明盒子	不透明盒子
依据	详设/源码	SRS/概设
方法	逻辑覆盖、基本路径	等价类、边界值、因果图、错误推测
阶段	主要单元测试	集成/确认/系统/验收
优点	能构造特定数据测试程序元素；有充分性度量；工具多	适用各阶段；从功能角度；易入手
缺点	用例设计难；无法测未实现规格部分；工作量大，通常只用于单元测试	部分代码测不到；规格说明有误则无法发现；不易充分性测试

3.2 6 种逻辑覆盖强弱

语句 < 判定 < 条件 < 判定-条件 < 条件组合 / 多重条件 < 路径。
注意：“路径覆盖”是路径维度的最强；“条件组合覆盖”是条件组合维度的最强；二者侧重点不同，需综合使用。

3.3 5 个测试阶段对比

阶段	测试对象	测试依据	主要方法	主导人员
单元测试	基本单元（类/函数/模块）	详细设计说明书	白盒为主，黑盒为辅	开发人员
集成测试	组装后的模块群、模块接口	概要设计说明书	黑盒为主	开发 + 测试
确认测试	基本可运行的系统	需求规格说明书 SRS	黑盒	专职测试人员
系统测试	真实运行环境下的整个系统	需求规格说明 + 合同 + 环境	黑盒	甲乙双方测试组
验收测试	试运行后稳定的系统、核心功能	用户合同 + 验收标准	黑盒；α/β	用户主导

3.4 Error / Defect / Fault / Failure

术语	中文	本质	发生阶段
Error	错误	人为差错	需求/设计/开发
Defect	缺陷	静态偏差	存在于软件中
Fault	故障	动态内部状态	运行时被激活
Failure	失效	外部不可接受状态	运行时呈现给用户

四、开放题答题模板

Q1 软件测试的目的？分别从使用者和开发者角度描述。

使用者角度：① 通过测试暴露软件中的错误和缺陷，以考虑是否可接受该产品；② 验证软件的功能与性能是否满足用户需求；③ 为用户提供接受/拒绝的客观依据；④ 通过验收测试（含 α/β）发现实际使用中的问题，建立使用信心。

开发者角度：① 以最少的人力、物力、时间找出各种错误和缺陷，并通过修正提高软件质量；② 验证软件已正确实现用户需求；③ 验证算法正确性、性能指标、设计假设；④ 每一次测试活动的目的是发现至今尚未被发现的错误，而不是证明系统正确。

Q2 开展软件测试所必须具备的三个条件？

解读一（流程视角，推荐）：① **测试依据/软件配置**——已评审通过的《需求规格说明书》及设计文档、源程序；② **测试用例/测试配置**——根据需求规格设计的、覆盖全部测试范围的用例集合（输入、操作、预期结果），并经评审纳入配置管理；③ **测试环境与工具**——搭建好的软硬件、数据环境及必要测试工具、错误跟踪平台。

解读二（可测试性视角）：① **可用性**——系统错误少、无阻碍测试执行的问题；② **可观察性**——每个测试有唯一明确输出、状态可见；③ **可控制性**——所有可能的输出都能由某种输入组合产生。

Q3 软件测试的步骤？各阶段的测试对象和依据？

5 步：① **单元测试**，对象 = 基本单元（类/函数/模块），依据 = 详细设计；② **集成测试**，对象 = 组装后的模块群与模块接口，依据 = 概要设计；③ **确认测试**，对象 = 基本可运行的系统，依据 = 需求规格说明 SRS；④ **系统测试**，对象 = 真实运行环境下的整个系统，依据 = 需求规格说明 + 合同；⑤ **验收测试**，对象 = 试运行后稳定的系统、面向用户的核心功能，依据 = 用户合同/验收标准。

Q4 基本路径法大题模板

四步走（背）：① **画控制流图**：先把程序流程图中的复合条件拆成单条件嵌套判定；顺序结点合并；分支汇聚处加虚拟汇聚结点。② **算 V(G)**（列出 3 种公式，任选代入）：
• 区域数法： $V(G) = \text{区域数}$ （含图形外开放区域）
• 欧拉公式： $V(G) = E - N + 2$ （E=边数，N=结点数）
• 判定结点法： $V(G) = P + 1$ （P=判定结点数）③ **写独立路径集**：条数 ≤ V(G)；每条至少含 1 条其他路径未走过的边。
④ **设计用例**：对每条路径代入输入数据，给出预期结果。

Q5 等价类划分大题模板

三步走（背）：① **梳理输入条件**：从需求规格中找出所有输入字段及其限制（区间、集合、布尔、规则等）。② **列等价类表**：表头 = 输入条件 / 有效等价类 / 编号 / 无效等价类 / 编号；按 5 条划分原则填写。③ **设计测试用例**：给每个等价类编号；有效用例尽量合并（1 个用例覆盖多个有效等价类）；无效用例逐个独立（每次只覆盖 1 个无效等价类）。

Q6 集成测试发现问题的修复流程（教材 10.3.5~10.3.7）

① 记录错误（状态 New）→ ② 错误确认与分配（Open / Declined / Deferred）→ ③ 定位与诊断（接口数据、模块副作用、全局数据、误差累积）→ ④ 修改代码 → ⑤ 单元测试验证 → ⑥ 重新集成 → ⑦ 回归测试（通过 Closed，未过 ReOpen）→ ⑧ 更新文档与配置 → ⑨ 测试总结报告。

Q7 软件维护阶段是否还需要测试？

需要。交付后进入维护阶段，每次修改（改正性/适应性/完善性/预防性）都须进行回归测试，验证修改未引入新错误且原有功能未失效。“交付后不再测试”是错误表述。

五、历年真题汇总与详解

5.1 判断题

【2007–2008 期末 A】软件质量主要通过软件的功能测试来保证。

×。质量保证依赖完整开发过程活动：需求/设计评审、编码规范、SQA、以及完整的测试流程（单元/集成/确认/系统/验收）。功能测试只是其中之一；非功能质量（性能、可靠性、安全性、可维护性）须专门测试与全生命周期质量活动保证。

【2007–2008 期末 A】单元测试中只能使用白盒测试方法。

×。教材 10.2.1：单元测试“以白盒为主、黑盒为辅”。边界测试、错误处理测试中常采用黑盒方法（边界值分析、等价类划分）。

【2008–2009 期末 A / 2009–2010 期末 A】采用黑盒测试系统功能时，完全不需要了解程序内部结构。

√。黑盒测试把程序看作不能打开的黑盒子，仅依据需求规格说明，关注输入输出对应关系，不依赖内部结构、控制流、数据流。

【2008–2009 期末 A / 2009–2010 期末 A】软件测试的目的证明提交的软件是正确的。

×。教材 10.1.3：“每一次测试活动的目的是发现至今尚未被发现的错误，**而不是证明系统是正确的**。”穷举测试不可能，无法证明软件完全正确。

【2010–2011 期末 B / 2011–2012 期末 B】通过软件测试，可以表明软件中不存在错误。

×。Dijkstra 名言："测试只能证明缺陷存在，不能证明缺陷不存在。"穷举测试不可能，因此测试通过 ≠ 软件无错。

【2010–2011 期末 B】软件实现包含单元测试。

√。广义软件实现 = 详细设计 + 编码 + 单元测试 + 集成测试；狭义 = 编码 + 单元测试。两者均含单元测试。

【2010–2011 期末 B】黑盒测试不能应用穷举法，白盒测试可以应用。

×。黑盒与白盒都不能应用穷举法。黑盒输入域天文数字；白盒因循环路径数指数增长，路径覆盖几乎不可能。

【2012–2013 期末 B】一个成功的软件测试表示查出了系统中所有的错误和缺陷。

×。成功的测试 = 发现了至今未发现的错误。穷尽测试不可行，"查出所有错误"不可能达到。

【2012–2013 期末 B】白盒测试中条件–判定组合逻辑覆盖方法是最严谨的。

×。最严谨的是路径覆盖。判定–条件覆盖（条件–判定组合）存在"某些条件会掩盖另一些条件判断"的缺陷（短路求值）。

【2013–2014 期末 A】软件测试用例由一系列满足功能成功执行的测试输入数据和预期的输出结果组成。

×。测试用例不限于"功能成功执行"——还必须包含失败场景、异常路径、边界条件。基本内容 = 用例编号、名称、输入、操作过程、预期结果等。

【2013–2014 期末 A】白盒测试中路径覆盖加上条件–判定组合逻辑覆盖可以解决所有判定结构的问题。

×。即便综合两者，仍不能"解决所有问题"——例如循环结构、复合判定中的短路求值、条件组合的覆盖完整性等都可能漏测。

【2018–2019 期末 B】确认测试是以测试人员为主，开发人员为辅，并且以系统设计说明书为参照标准的测试阶段。

×。确认测试的参照标准是需求规格说明 SRS，不是系统设计说明书。"以测试人员为主"无误，但参照文档错位即整体判错。系统设计说明书是集成/单元测试依据。

【2018–2019 期末 B】在基于等价类划分法设计测试用例时，应该使得每个测试用例尽可能多地对无效等价类进行覆盖。

×。规则恰好相反：有效等价类"尽可能多覆盖"（1 用例覆盖多个），无效等价类"每次只覆盖一个"。否则当一个用例检测到 1 个错误就停止，会掩盖其他错误。

【2023–2024 期末 A】系统的确认测试表示要通过基于需求分析规格说明书的测试用例。/ 软件交付用户之后就不再需要进行软件测试活动了。

前者√——确认测试以 SRS 为依据；后者×——交付后维护阶段每次修改都须回归测试。

5.2 单项选择题

【2007–2008 期末 A】测试的关键问题是（ ）

D。如何选择测试用例。穷举不可能，核心在于用最少用例发现最多错误。

【2007–2008 期末 A】软件测试的目的是（ ） A. 表明程序没有错误 B. 说明程序能正确执行 C. 发现程序中的错误 D. 评价程序的质量

C。A、B 与测试原则相反；D 是副产物，非主要目的。

【2007–2008 期末 A】用白盒测试法设计测试用例的方法包括（ ） A. 错误推测 B. 因果图 C. 基本路径测试 D. 边界值分析

C。其余三项（错误推测、因果图、边界值分析）均为黑盒方法。

【2008–2009 期末 A】软件测试内容不包括（ ）

对代码进行调试。调试（Debugging）属开发活动（定位并修复错误），与"测试 = 发现错误"是不同活动。

【2008–2009 期末 A】白盒测试法中最强的逻辑覆盖是（ ）

A. 多重条件覆盖 B. 判定覆盖 C. 路径覆盖 D. 语句覆盖
C 路径覆盖。路径维度最强，要求覆盖所有从入口到出口的路径。

【2009–2010 期末 A】黑盒测试在设计测试用例时，主要需要研究（ ）

需求规格说明与概要设计说明。详细设计是白盒依据；项目开发计划是管理文档。

【2009–2010 期末 A】用白盒技术设计测试用例的方法是（ ） A. 错误推测 B. 因果图 C. 基本路径测试 D. 边界值分析

C。同 2007 题。

【2010–2011 期末 B】以下不属于黑盒测试技术的是（ ） A. 边界值分析 B. 等价类划分 C. 基本路径测试 D. 因果图

C。基本路径测试属白盒。

【2011–2012 期末 B】软件集成测试的对象是（ ） A. 软件代码 B. 详细设计说明书 C. 需求分析规格说明书 D. 用户需求说明书

官方答 B（但与教材有偏差，争议题）。教材本意：集成测试对象 = 已通过单元测试、按概要设计组装起来的模块群（接口）；依据 = 概要设计说明书。选项无"模块群"或"概要设计说明书"。建议按教材"模块群/接口"作答；若被迫四选一，官方取 B（详设）实为单元测试依据，存疑。

【2012–2013 期末 B】以下白盒测试技术是必须执行的（ ）

A. 语句覆盖 B. 路径覆盖 C. 判定覆盖 D. 条件覆盖

官方 B（路径覆盖）。按"作为完备测试目标必须做到"理解选 B；若按"最低基本要求"理解，教材亦指出"语句覆盖是设计测试用例的最基本要求"，可选 A。本题存在歧义。

【2013–2014 期末 A】一段简单的赋值操作的程序必须执行的白盒测试是（ ） A. 语句覆盖 B. 条件–判定组合覆盖 C. 判定覆盖 D. 条件覆盖

A。赋值不含判定/条件，无判定节点，其余覆盖不适用。语句覆盖是任何程序（含纯顺序结构）的最基本要求。

【2013–2014 期末 A】黑盒测试因果图的目的是（ ） A. 绘制因果图 B. 确定测试用例 C. 通过原因确定结果 D. 减少测试用例的数目

D。因果图通过约束符号去掉不可能组合、合并冗余列，从而有效减少测试用例数目。B 是直接产出，但题目问"目的"，D 更贴切。

【2023–2024 期末 A】某程序使用等价类划分构造测试用例，有效等价类 3 个，无效等价类 4 个，最少需要多少组？（ ）

A. 7 B. 5 C. 4 D. 11

B（5 组）。1 组覆盖全部 3 个有效 + 4 组各覆盖 1 个无效 = 5 组。

【2023–2024 期末 A】以下哪一项方法可以有效降低测试用例的数量？（ ） A. 因果图 B. 基本路径测试 C. 边界值分析 D. 等价类划分

A 因果图。因果图 + 判定表能化简不可能组合、合并等价值，减少用例数。边界值是在等价类基础上"增加"用例。

5.3 填空题

【2023–2024 期末 A】测试用例由测试的（ ）和（ ）构成。

输入数据（输入条件）与预期输出结果。

5.4 简答题

【2007–2008 / 2008–2009 / 2010–2011 期末】简述软件测试的步骤及各阶段的测试对象和依据。

步骤	对象	依据	方法/人员
单元测试	基本单元	详细设计	白盒为主，开发
集成测试	模块群、接口	概要设计	黑盒，开发+测试
确认测试	基本可运行系统	SRS	黑盒，专职测试
系统测试	真实环境下的整个系统	需求+合同+环境	黑盒，甲乙双方
验收测试	试运行后稳定系统、核心功能	用户合同/验收标准	用户主导，α/β

【2018–2019 / 2021–2022 期末】请给出执行软件测试所必须具备的三个条件。

方案一（流程视角）：① 测试依据（SRS、设计文档、源程序）；② 测试用例（覆盖全部测试范围、含输入/操作/预期结果）；③ 测试环境（硬件、软件、数据环境）与工具。方案二（可测试性）：① 可用性；② 可观察性；③ 可控制性。

【2021–2022 期末 A】如果在集成测试中发现了问题，开发人员需要做哪些必要活动才能通过该集成测试？

① 记录错误（New）→ ② 错误确认与分配（Open/Declined/Deferred）→ ③ 定位与诊断（接口数据/副作用/全局数据/误差累积）→ ④ 修改代码 → ⑤ 单元测试验证 → ⑥ 重新集成 → ⑦ 回归测试（Closed/ReOpen）→ ⑧ 更新文档与配置。

【2023–2024 期末 A】分别从使用者和开发者的角度描述软件测试的目的。

使用者：① 暴露错误和缺陷以决定是否接受；② 验证功能与性能满足需求；③ 提供接受/拒绝的客观依据；④ 通过验收测试（含 α/β）发现实际使用中的问题。开发者：① 以最少人力物力时间找出错误并修正，提高质量；② 验证算法、性能、设计假设；③ 缺陷分布作为过程改进依据；④ 每一次测试活动的目的是发现尚未发现的错误，而不是证明系统正确。

5.5 应用题（大题）

【2007–2008 期末 A】输入年、月、日判断下一天。年份 1900–2099；月份 1–12；日 1–31。请按等价类划分原则给出等价类表。

输入	有效	编号	无效	编号
年份	$1900 \leq \text{year} \leq 2099$	1	$\text{year} < 1900$ / $\text{year} > 2099$ / 非整数	10/11/12
月份	$1 \leq \text{month} \leq 12$	2	$\text{month} < 1$ / $\text{month} > 12$ / 非整数	13/14/15
日	$1 \leq \text{day} \leq 31$	3	$\text{day} < 1$ / $\text{day} > 31$ / 非整数	16/17/18
三者关系	符合公历历法（如 4 月 31 日无效、闰年 2 月 29 日有效）	4~9	违反历法（如 2 月 30 日、4 月 31 日、非闰年 2 月 29 日）	19~21

说明：等价类表不唯一，须给出唯一编号，并按区间/集合/规则 5 条原则划分。

【2008–2009 期末 A】某城市电话号码：地区码 4 位、首位为 0；前缀 3 位、首位非 0/1；后缀 4 位。划分等价类。

输入	有效	编号	无效	编号
地区码	4 位数字且首位为 0	1	非 4 位 / 首位非 0 / 含非数字	7/8/9
前缀	3 位数字且首位非 0/1	2	非 3 位 / 首位为 0 / 首位为 1 / 含非数字	10/11/12/13
后缀	4 位数字	3	非 4 位 / 含非数字	14/15

【2009–2010 期末 A】世博会门票检测码：标识码 = "2010EXPO"；内容码 = 1000~9999 四位数字；验证码 = 以 "VF" 为前缀的十位数字。划分等价类。

输入	有效	编号	无效	编号
标识码	等于 "2010EXPO"	1	≠ "2010EXPO" / 含非英数 / 长度≠8	8/9/10
内容码	1000~9999 四位数字	2	<1000 / >9999 / 含非数字 / 长度≠4	11/12/13/14
验证码	以 "VF" 前缀的 10 位	3~5	前缀≠"VF" / 总长≠10 / 含非数字	15/16/17

【2010–2011 期末 B】团购验证码：类型码 ∈ {"00","23","79","96"}；内容码 10000~99999 五位数字；检验码 = "11" 前缀 + "99" 后缀的 10 位数字。划分等价类。

输入	有效	编号	无效	编号
类型码	取 4 个指定值之一	1~4	非指定值 / 含非数字 / 长度≠2	N1/N2/N3
内容码	10000~99999 五位数字	5	<10000 / >99999 / 含非数字 / 长度≠5	N4~N7
检验码	10 位数字、以 11 开头、以 99 结尾	6~8	前缀≠11 / 后缀≠99 / 长度≠10 / 含非数字	N8~N11

【2011–2012 期末 B】平面两点 a(x1,y1)、b(x2,y2)：a 在第一象限；b 在 x 轴；ab 距离大于 3 且不超过 9。划分等价类。

输入	有效	编号	无效	编号
a 点	$x1 > 0$ 且 $y1 > 0$	1	$x1 \leq 0$ / $y1 \leq 0$ （其他象限或坐标轴）	2/3/4
b 点	$y2 = 0$ （x 轴）	5	$y2 \neq 0$	6
ab 距离 d	$3 < d \leq 9$	7	$d \leq 3$ / $d > 9$	8/9

评分要点：输入条件 1 分；有效 3 分；无效 6 分（每个 1 分）；缺少编号扣 1 分；没有以坐标作为输入条件描述先扣一半。

【2012–2013 期末 B】登录验证码：标识码 = "11952799"；内容码 = $100 \leq x < 999$ 或 $10000 < x \leq 99999$ ；检验码 = "11" 开头 + "99" 结尾的 8 位数字，且 ≠ 标识码。划分等价类。

输入	有效	编号	无效	编号
标识码	"11952799"	1	≠ "11952799" / 含非数字 / 长度≠8 非 11952799 的数字即可，不用这么麻烦	N1~N3
内容码	$100 \leq x < 999$ 或 $10000 < x \leq 99999$	2,3	<100 / $999 \leq x \leq 10000$ / >99999 / 含非数字	N4~N7
检验码	8 位数字、"11"开头、"99"结尾、≠标识码	4~7	前缀≠"11" / 后缀≠"99" / 长度≠8 / =标识码 / 含非数字	N8~N12

形如 11xxxx99 的数字，xxxx 取值范围为 0000~9999，且 xxxx 不等于 9527(8)

11952799(9)
以 "11" 为前缀 但不以 "99" 为后缀的数字 (10)
以 "99" 为后缀 但不以 "11" 为前缀的数字 (11)
既不以 "11" 为前缀 也不以 "99" 为后缀的数字 (12)
以 "11" 为前缀 并且以 "99" 为后缀的非 8 位数字 (13)

【2021-2022 期末 A】购物网站搜索：商品名称（≤10 字符、非空）、最低价格（≥0、≤最高）、最高价格（≥0、≥最低）、最早日期（YYYYMMDD、≤最晚）、最晚日期（YYYYMMDD、≥最早）。划分等价类。

输入	有效	编号	无效	编号
商品名称	长度 1-10 字符	1	空 / 长度>10 / 含非法字符	N1~N3
最低价格	≥0 且 ≤ 最高	2	<0 / >最高 / 非数字	N4~N6
最高价格	≥0 且 ≥ 最低	3	<0 / <最低 / 非数字	N7~N9
最早日期	YYYYMMDD 合法且 ≤ 最晚	4	格式错误 / >最晚 / 非法日期 (2 月 30 等)	N10~N12
最晚日期	YYYYMMDD 合法且 ≥ 最早	5	格式错误 / <最早 / 非法日期	N13~N15

【2012-2013 期末 B / 大题】20 个员工工资核算：正式工投诉 <3 不扣、≥3 每次扣 50；临时工投诉 <2 每次扣 50、≥2 每次扣 100。基本路径法。

① 程序流程图略；② 控制流图：含 4 个判定节点（员工循环、正式/临时、投诉次数阈值、是否有投诉）。③ V(G) 计算：区域数法 V(G)=5；欧拉公式 V(G)=E-N+2=13-10+2=5；判定节点法 V(G)=P+1=4+1=5。④ 独立路径集（5 条）：

Path1: A → J (无员工)
Path2: A → B → C → D → I → J (正式工, 投诉<3, 不扣)
Path3: A → B → C → D → E → I → J (正式工, 投诉≥3, 扣 50)
Path4: A → B → C → F → G → I → J (临时工, 投诉<2, 扣 50)
Path5: A → B → C → F → H → I → J (临时工, 投诉≥2, 扣 100)

评分要点：关键 4 个判断节点少 1 个扣 0.5；直接给 V(G) 无计算说明扣 0.5；独立路径条数 ≤ 5，每条至少含 1 条新边，错 1 个扣 0.5。

【2013-2014 期末 A】已知某程序流程图。① 将复合条件判定变为单条件嵌套判定；② 导出控制流图、计算 V(G)；③ 给出独立路径集。

① 拆分：形如 (A>1) and (B=0) → 先判 A>1 (真则继续判 B=0)，(A=2) or (X>1) → 先判 A=2 (真则短路整体真，假则继续判 X>1)。② 控制流图：4 个判定结点；V(G)=5 (区域数 5 / E-N+2=12-9+2=5 / P+1=4+1=5)。③ 独立路径集（5 条）：

Path1: 1 → 9
Path2: 1 → 2 → 3 → 7 → 8 → 1 → 9
Path3: 1 → 2 → 3 → 4 → 6 → 8 → 1 → 9
Path4: 1 → 2 → 3 → 4 → 5 → 6 → 8 → 1 → 9
Path5: 1 → 2 → 3 → 4 → 5 → 7 → 8 → 1 → 9

【2011-2012 期末 B】已知某带循环嵌套的程序流程图。① 穷举路径数；② V(G)；③ 独立路径集。

① 穷举路径数：内循环 3 次、每次 2 选择 = $2^3 = 8$ ；外循环 5 次、每次 10 选择 = 10^5 ；总数 = 8×10^5 (仅作参考估算示例)。② V(G)=6 (区域数 6 / E-N+2=17-13+2=6 / P+1=5+1=6)。③ 独立路径集（6 条）：

Path1: 1→13
Path2: 1→2→3→7→12→1→13
Path3: 1→2→3→4→5→12→1→13
Path4: 1→2→3→4→6→12→1→13
Path5: 1→2→3→4→6→8→9→11→6→12→1→13
Path6: 1→2→3→4→6→8→10→11→6→12→1→13

【2018-2019 期末 B】int Test(int l_count, int l_flag) — while (l_count>0), 内含 if(0==l_flag) break; else if(1==l_flag) ... else ...。要求修改流程图、控制流图、V(G)、独立路径集。

① 拆分复合条件：题目内每个判定已是单条件，按“嵌套”形式画即可，注意 if-else if-else 多分支汇聚到一个虚拟结点。② 控制流图与 V(G)：3 个判定结点 (while、if l_flag==0、else if l_flag==1) → V(G)=P+1=4 (也可区域数法 / E-N+2 验证)。③ 独立路径集（4 条）：

Path1: 入口 → while(假) → return (l_count≤0)
Path2: 入口 → while(真) → l_flag==0 → break → return
Path3: 入口 → while(真) → l_flag≠0 → l_flag==1 → l_temp*10 → l_count-- → while → return
Path4: 入口 → while(真) → l_flag≠0 → l_flag≠1 → l_temp*20 → l_count-- → while → return

【2021-2022 期末 A】void Func(int a,int b,int c) — while(a>0), 含 (b>1)&&(c>2) 与 (b<-1)||(c<-2) 两个复合判定。① 画流程图（复合判定拆为单条件嵌套）；② 控制流图 + V(G)；③ 独立路径集。

① 复合条件拆分：
• (b>1) && (c>2) → 先判 b>1 (真则继续判 c>2，两者都真才进 break 分支)
• (b<-1) || (c<-2) → 先判 b<-1 (真则整体真；假则继续判 c<-2) ② 控制流图：共 5 个判定节点 (while + 拆分后 2×2 个单条件判定)。V(G)=P+1=6 (区域法/E-N+2 一致即可)。③ 独立路径集（6 条）：

Path1: 入口 → while(假) → 出口
Path2: 入口 → while(真) → b>1=Y → c>2=Y → break → 出口
Path3: 入口 → while(真) → b>1=Y → c>2=N → b<-1 → c<-2 → x=(y+4)(z+15); a-- → while → 出口
Path4: 入口 → while(真) → b>1=N → b<-1=Y → x=(y+2)(z+10); a-- → while → 出口
Path5: 入口 → while(真) → b>1=N → b<-1=N → c<-2=Y → x=(y+2)(z+10); a-- → while → 出口
Path6: 入口 → while(真) → b>1=N → b<-1=N → c<-2=N → x=(y+4)(z+15); a-- → while → 出口

【2023-2024 期末 A】Python 折扣函数：if role==1 → 0.8; elif role==2 and date=='0910' → 0.8; elif price>=200 → 0.85; elif role==2 → 0.9; else → price。① 控制流图；② V(G)；③ 必须覆盖的基本路径集。

① 控制流图：4 个判定结点 (if role==1; elif role==2 and date=='0910', 复合条件需拆为两步; elif price>=200; elif role==2)。② V(G)=P+1=5 (拆分后 P=4，若不拆分按表面 4 个 elif 判定也是 V(G)=5；区域法一致即可)。③ 基本路径集（5 条）：

P1: role==1 → return 0.8×price
P2: role==2 ∧ date=='0910' → return 0.8×price
P3: 不满足上两条 ∧ price>=200 → return 0.85×price
P4: 不满足上三条 ∧ role==2 → return 0.9×price
P5: 其余 → return price

【2009-2010 / 2010-2011 期末】基本路径法典型大题（中央空调主控机、电子商务 10 笔交易打折）。

通用答题套路：① 由题意画程序流程图（判定节点对应角色/金额/次数阈值）；② 拆分复合条件 → 画控制流图；③ 三种公式任一代入算 V(G) (推荐 P+1 最稳)；④ 写出 ≤ V(G) 条独立路径，每条至少 1 条新边。

示例 (10 笔交易打折，普通顾客 <500 不折/≥500 打 8 折；会员 <1000 打 7 折/≥1000 打 6 折)：判定结点 4 个 (循环、是否普通、金额阈值、是否会员) → V(G)=5；独立路径 5 条 = {无交易 / 普通<500 / 普通≥500 / 会员<1000 / 会员≥1000}。

六、A4 双栏 Cheat Sheet（背）

测试目的（背）

- 用户：暴露错误，决定接受与否
- 开发者：发现错误并修正，验证需求
- Myers：发现至今未发现的错误；不能证明无错
- 测试 ≠ 调试；测试 = 发现，调试 = 修复

测试 7 原则

- 尽早不断测试
- 用例 = 输入 + 预期输出
- 程序员避免自测
- 合理 + 不合理输入
- 群集现象
- 严格按计划，全面检查
- 穷举不可能；文档保存

白盒 vs 黑盒

- 白盒：看内部 / 详设 / 逻辑覆盖 + 基本路径 / 单元
- 黑盒：看外部 / SRS / 等价类 + 边界值 + 因果图 / 集成~验收
- 白盒：模块独立路径、判定真假、循环边界、内部数据结构
- 黑盒：功能、接口输入输出、性能、初始化/终止

6 种逻辑覆盖（强弱递增）

语句 < 判定 < 条件 < 判定-条件 < 条件组合 / 路径

- 语句：每条至少 1 次（最弱）
- 判定：真假分支各 1 次
- 条件：每个条件真假各 1 次
- 判定-条件：兼顾两者
- 条件组合：所有组合各 1 次
- 路径：所有路径各 1 次（最强但循环不可行）

基本路径法 4 步

1. 流程图 → 控制流图（拆复合条件 / 合并顺序结点 / 加汇聚结点）
2. 算 $V(G)$ ：区域数 / $E - N + 2$ / $P + 1$
3. 独立路径集（条数 $\leq V(G)$ ，每条至少 1 条新边）
4. 设计用例：输入数据 + 预期结果

等价类划分

- 区间 $[x,y]$ ：1 有效 + 2 无效
- 集合：1 有效 + 1 无效
- 布尔：1 真 + 1 假
- 离散值分别处理：每值 1 有效 + 1 无效
- 规则：1 符合 + N 违反
- 设计：有效尽量合并 / 无效逐个独立
- 最少用例 = 1 + 无效等价类数

边界值

- 取正好等于、刚刚大于、刚刚小于边界
- 个数： $n-1$ 、 n 、 $n+1$ 、 $m-1$ 、 m 、 $m+1$
- 有序集：第一个、最后一个
- 补充等价类的不足

因果图

- 关系：恒等 $\equiv$ / 非 $\neg$ / 或 $\vee$ / 与 $\wedge$
- 约束：E 互斥 / I 包含 / O 唯一 / R 要求 / M 屏蔽
- 步骤：找因→果 → 画图 → 标约束 → 判定表 → 用例
- 目的：减少测试用例数（剔除不可能组合）

5 个测试阶段

- 单元 → 详设 / 白盒 / 开发者 / 驱动+桩
- 集成 → 概设 / 黑盒 / 开发+测试 / 自顶向下用桩、自底向上用驱动
- 确认 → SRS / 黑盒 / 专职测试
- 系统 → 真实环境 / 黑盒 / 甲乙双方
- 验收 → 用户合同 / 用户主导 / $\alpha+\beta$

驱动 vs 桩

- 驱动 driver：模拟主程序，调用被测模块
- 桩 stub：模拟子模块，被被测模块调用
- 自顶向下 → 大量桩
- 自底向上 → 大量驱动
- 三明治 → 既需驱动又需桩

α vs β

- α ：开发环境、开发者旁观、首批用户、FURPS
- β ：实际使用环境、用户独立、广大用户、最后阶段

术语链

Error（人为差错）→ Defect（静态偏差）→ Fault（动态内部状态）→ Failure（外部失效）

错误状态机

New → Open → Fixed → Closed / ReOpen; Declined; Deferred

回归测试

- 修改后必须执行
- 策略：全回归 / 风险 / 频度 / 影响分析
- 交付后维护仍需

可靠性

- 可靠性 = $[0,t]$ 未失效概率
- 可用性 = 时刻 t 成功运行概率
- $MTTF = 1 / [K \cdot (ET/IT - Ec(t)/IT)]$
- $K \approx 200$
- Hyman: $B0 = B1 \cdot B2 / bc$

大题避坑

- $V(G)$ 必须给出至少 1 种计算说明
- 独立路径 $\leq V(G)$ ，每条至少 1 新边
- 等价类必须编号
- 无效等价类每个用例只覆盖 1 个
- 判定节点不能少，少 1 个扣 0.5
- 复合条件必须拆开